Google Reichman University | AI Tech School

Project Breakdown

or: you expect me to do WHAT?!

May 2026

The Process

What?

Ideation

- What is a potential use case?
- Who is the potential client / stakeholder?
- What pain points are you solving?
    i. (== What pain points are you solving?)
- Examples for inspiration - from previous cohort

What?

Define a POC

- Define inputs & outputs
- Define success metrics (e.g., accuracy)
- Create test set(s) to measure success
- Goal: reach **xx% success** on your data

What?

Implement that POC

- Purpose: show feasibility
- Just get something working
- Start with strong models & simple prompts
- After initial results, check if the goal is realistic
- Don’t obsess over a single example
- If results are far below target after initial testing, it's okay to simplify the problem scope — document why

What?

Test at scale

- Measure performance on a larger test set
- What counts as "scale" here? Aim for at least *[X]* examples
- Manual evaluation is fine for small sets — automate only if it makes sense for your use case

What?

Analyze the results and see places of improvement

- Your POC works! Now evaluate:
  1. Latency
  2. Cost
  3. Quality aspects (issues not addressed in the POC)
  4. Other considerations

What?

Improve at least one dimension (latency / cost / quality / robustness)

- Use cheaper/faster models
- Add processing steps to improve weak points
- You don't need to improve everything. Pick the most impactful lever given your use case and justify why.

Example

or: a base you can start from

Documentation Q&A

0. Rough description:

Build a chatbot / Q&A engine based on a company’s technical documentation

Documentation Q&A

1. Problem Selection & Definition

- We want to answer the following:
  i. Industry/Domain: *Customer Support*
  ii. Current Processes or Systems: *Support agents manually search docs → time-consuming, inconsistent*
  iii. Pain Points:
    1. *Slow responses*
    2. *Inconsistent quality*
    3. *Agents overwhelmed with information*
  iv. Business Impact: *Faster, more reliable answers → improved customer satisfaction & reduced support costs*

Documentation Q&A

1.5. Define a POC

- Inputs: *Customer questions*
- Outputs: *Concise answers sourced from documentation*
- Data: *public documentation + questions*
- Metrics:
  1. *Accuracy of answers (how do you measure it?)*
  2. *% of questions answered within scope*
  3. *Grounding*
- Goal: *Achieve 70–80% accuracy on test questions*

Documentation Q&A

2. Market Research & Technical Discovery

- What are the components needed?
  i. Data scraping
  ii. Vecdor DB
  iii. Generator
- Are there out-of-the-box solutions?
  i. At least consider (even if you want to implement yourself)

Documentation Q&A

3. Implement the POC

- Scrape all the documentation
    i. You can checkout BrightData, LlamaParse, …
- Chunk documentation into embeddings
- Build a retrieval pipeline
- Connect to an LLM for answer generation
- Test on sample support questions

Documentation Q&A

4. Analyze the POC

- Measure performance on test questions
- Identify weak points:
  i. Are certain topics under-covered?
  ii. Are answers too verbose or hallucinated?
  iii. Is latency acceptable?
  iv. Assess cost per query
- What do you do when there's a new document?

Documentation Q&A

5. Take a step towards production

- Try smaller, faster models for cheaper inference
- Add answer validation (e.g., “quote the source”)
- Experiment with better chunking strategies
- Add a fallback system for unanswered questions

The final stage.

You built it. Now tell the story.

The Goal

- Show us what you actually did and why it’s interesting.
- Don’t think of it as a final project in a course, but a project you did for work.
- We want to see:
  - What you built
  - How it works
  - Why it’s useful

The Presentation

- ⏱️ 8-minute presentation + 4-minute Q&A
- Aim for 6 minutes when you practice
- For pairs: divide your presentation into two parts and stick with them. Back and forth will only waste time.
- Use large fonts (24+), less text, informative graphs/data visualizations
- If you plan to live demo: make sure you have a video as backup

The Flow (a.k.a. What We Want to See)

- The problem: “Customer support answers take too long.”
- The solution: “We built a RAG-based Q&A bot using company docs.”
- Architecture: Diagram + short explanation of how data flows.
- Evaluation: “Our system answers 80% of questions correctly.”
- Impact: “Reduced average handling time by 60%.”
- Demo: A short clip, a live query, or screenshots.
- Next steps: “We’d like to add source validation and faster models.”

Technical & Production Considerations

- Tech Stack: Which models, tools, or frameworks did you use — and why those?
- Trade-offs: What compromises did you make (accuracy vs latency, quality vs cost)?
- Simplicity is fine: Clear, working > complex and fragile.
- Who’s it for? Who actually benefits from your system?
- Real-world benefit: Show tangible value — faster answers, lower cost, better user experience.
- Production mindset: How would this run in the real world? (Think latency, cost, security, scalability.)
- Next steps: If you had two more weeks or a small budget, what would you build or improve?

Technical & Production Considerations

This is an example of a bad slide in a presentation

?

ɜtter user experience.

orld? (Think latency, cost,

budget, what would you build or improve?

What NOT to Do

- Open with a long story about "the age of AI."
- Read your slides word-for-word.
- Show endless code or notebooks.
- Skip evaluation ("it works" isn't proof).
- *Tip: keep optional slides in your presentation. If time is an issue, just skip them.*

Delivery Tips

- Think like you're presenting at work.
  - Use visuals — but only if they help explain something.
  - Speak naturally — not like you're reading a blog post.
  - Mention challenges and how you solved them.
  - Show progress — even partial success is valuable.
  - Treat us like your managers or investors:
  - Convince us why this project deserves more time, resources, or a real pilot.